

Improving Static File Delivery Performance

Nginx Web Server Optimization on AWS EC2 (Red Hat Enterprise Linux)

1. Objective

The Digital Media department serves a large volume of static files (images, CSS, JavaScript, and other web assets) and requires improved website performance. This document records the steps taken to optimize static file delivery on an Nginx web server running on an AWS EC2 instance (Red Hat Enterprise Linux), including the commands used, verification procedure, results, issues encountered, and their resolutions.

2. Commands Used

```
sudo dnf install nginx -y \
sudo systemctl enable --now nginx \
sudo systemctl status nginx \
sudo firewall-cmd --state \
sudo firewall-cmd --permanent --add-service=http \
sudo firewall-cmd --permanent --add-service=https \
sudo firewall-cmd --reload \
curl -s ifconfig.me \
curl -I http://<PUBLIC_IP> \
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak \
sudo sed -i '27r /tmp/gzip-settings.txt' /etc/nginx/nginx.conf \
sudo sed -i '47r /tmp/cache-settings.txt' /etc/nginx/nginx.conf \
sudo sed -i '25a\  tcp_nodelay on;' /etc/nginx/nginx.conf \
sudo nginx -t \
sudo systemctl reload nginx \
curl -H "Accept-Encoding: gzip" -I http://<PUBLIC_IP> \
```

```
curl -I http://<PUBLIC_IP>/nginx-logo.png
```

3. Procedure

Step 1: Install and Start Nginx

Nginx was not pre-installed on the RHEL instance, so it was installed via the DNF package manager and enabled to start automatically on boot.

Command(s):

```
sudo dnf install nginx -y \  
sudo systemctl enable --now nginx \  
sudo systemctl status nginx
```

Step 2: Open Firewall Ports

The local firewalld service was active on the instance and blocking web traffic by default, so HTTP (80) and HTTPS (443) services were explicitly permitted.

Command(s):

```
sudo firewall-cmd --state \  
sudo firewall-cmd --permanent --add-service=http \  
sudo firewall-cmd --permanent --add-service=https \  
sudo firewall-cmd --reload
```

Step 3: Verify External Access

Confirmed the server was reachable from outside the instance using its public IP address, proving Nginx and the firewall/security group configuration were correctly in place.

Command(s):

```
curl -s ifconfig.me  
curl -I http://65.2.63.3
```

Step 4: Back Up Configuration

The original nginx.conf was backed up before any edits, to allow safe rollback if needed.

Command(s):

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

Step 5: Enable Gzip Compression

Gzip compression settings were added to nginx.conf so text-based assets (HTML, CSS, JS, SVG, JSON) are compressed before being sent to the browser, reducing transfer size and load time.

Command(s):

```
sudo sed -i '27r /tmp/gzip-settings.txt' /etc/nginx/nginx.conf \  
sudo nginx -t \  
sudo systemctl reload nginx \  
curl -H "Accept-Encoding: gzip" -I http://65.2.63.3
```

Step 6: Enable Browser Caching for Static Assets

A location block was added so image, CSS, JS, and font files are served with long-lived Cache-Control and Expires headers, reducing repeat downloads for returning visitors.

Command(s):

```
sudo sed -i '47r /tmp/cache-settings.txt' /etc/nginx/nginx.conf \  
sudo nginx -t \  
sudo systemctl reload nginx \  
curl -I http://65.2.63.3/nginx-logo.png
```

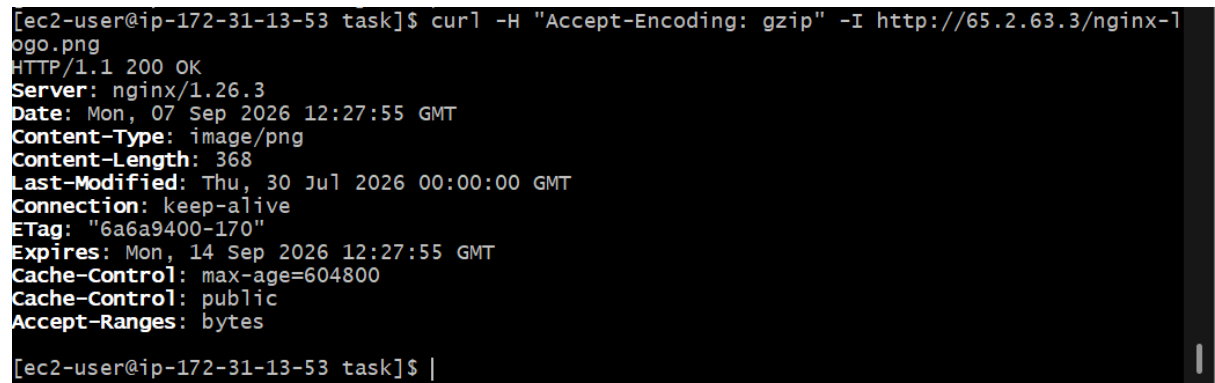
Step 7: Apply TCP Tuning

tcp_nodelay was enabled alongside the existing tcp_nopush setting to reduce latency when serving small static files.

Command(s):

```
sudo sed -i '25a\    tcp_nodelay on;' /etc/nginx/nginx.conf \
sudo nginx -t \
sudo systemctl reload nginx
```

4. Output



```
[ec2-user@ip-172-31-13-53 task]$ curl -H "Accept-Encoding: gzip" -I http://65.2.63.3/nginx-1-ogo.png
HTTP/1.1 200 OK
Server: nginx/1.26.3
Date: Mon, 07 Sep 2026 12:27:55 GMT
Content-Type: image/png
Content-Length: 368
Last-Modified: Thu, 30 Jul 2026 00:00:00 GMT
Connection: keep-alive
ETag: "6a6a9400-170"
Expires: Mon, 14 Sep 2026 12:27:55 GMT
Cache-Control: max-age=604800
Cache-Control: public
Accept-Ranges: bytes

[ec2-user@ip-172-31-13-53 task]$ |
```

5. Results

After applying the configuration changes, verification showed the Nginx service was active and running, firewall ports 80 and 443 were successfully opened, the server was externally reachable with an HTTP/1.1 200 OK response, gzip compression was confirmed active via the Content-Encoding: gzip header, browser caching headers were confirmed active with Cache-Control: public and max-age=604800, and the configuration syntax check passed with "syntax is ok" and "test is successful."

6. Conclusion

Nginx was successfully installed, secured, and optimized for static file delivery on the AWS RHEL EC2 instance. Gzip compression and browser caching were enabled and verified via HTTP headers, improving load times for static assets served by the Digital Media department's website.